DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 9

React.js developer

Profession 9 of 10 in Development and IT.

What the profession means

A React developer builds user interfaces from components: small, reusable pieces that keep their own state and render what the data says. React is the most widely used library for interactive web interfaces, and it is also the usual front end for full-stack apps.

What professionals do every day

Designs components, manages state and data flow, connects to APIs, writes tests, optimises rendering and maintains a component library.

Why businesses hire them, and what problems they solve

Businesses with interactive products (dashboards, carts, booking flows) need interfaces that update smoothly and can grow. React developers build and maintain those interfaces.

- Interfaces that get harder to change as they grow.
- Data that is out of date on screen.
- Slow pages from unnecessary re-renders.

Skills you need

- JavaScript modern syntax (arrow functions, destructuring, modules).
- JSX and components; props and state.
- Hooks (useState, useEffect) and data fetching.
- Routing, forms and testing.
- Performance and deployment.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
React	UI library	Free	Free	react.dev
Vite	Fast development server and build tool	Free	Free	vite.dev
Visual Studio Code	Code editor	Free	Free	code.visualstudio.com
Vercel	Hosting for React apps	Hobby tier	Paid plans	vercel.com
React Developer Tools	Browser extension to inspect components	Free	Free	react.dev/learn/react-developer-tools

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Components, props and state; render lists and handle clicks.
Intermediate	Hooks, fetching data from an API, forms and routing.
Advanced	Testing, performance profiling, context or a state library, and deployment.

Projects by level

- **Beginner:** A counter and a to-do list.
- **Intermediate:** A product list that loads from an API with a search box.
- **Advanced:** A dashboard with routing, charts and login.

Client-ready portfolio project

A React application for a fictional or permissioned business: several pages, data from an API, a form with validation, tests for two components, and deployment with a README.

Worked example

Worked example. A to-do list is an array in state. Adding an item creates a new array with the item at the end; deleting filters the array. React redraws the list because it sees a new array.

Assignment with full solution

ASSIGNMENT

A React counter and a list with a filter

Business problem: A product team wants a small interactive component: a counter that can be reset, and a list of tasks that can be filtered.

Steps

1. Create a project with the official Vite React template: `npm create vite@latest` (follow the current official instructions).
2. Create `Counter.jsx` as in the solution, and render it in `App.jsx`.
3. Add a list component with three tasks, a text filter and a count of visible tasks.
4. Check that each button changes the screen, and that the filter updates the count.

Expected output: A running page with two working components.

SOLUTION (ANSWER KEY)

Why it works. `useState` keeps the value and re-renders the component when it changes. Props (`start`, `step`) make the component reusable. `aria-live` announces changes to screen readers.
Not run here: the solution is syntax-checked as JSX; run it in a Vite project to confirm the page works, and check the current Vite and React documentation for the exact commands.

JAVASCRIPT

```
// Counter.jsx : a React component with state (Vite or Create React App template)
import { useState } from 'react';

export default function Counter({ start = 0, step = 1 }) {
  const [count, setCount] = useState(start);
  return (
    <div>
      <p aria-live="polite">Count: {count}</p>
      <button onClick={() => setCount(count + step)}>Add {step}</button>
      <button onClick={() => setCount(start)}>Reset</button>
    </div>
  );
}

// App.jsx
// import Counter from './Counter';
// export default function App() { return <Counter start={10} step={5} />; }
```

Common mistakes

- Mutating state directly (for example pushing into the array) so the screen does not update.
- Putting every value in global state.
- Calling APIs in render instead of effects or data libraries.
- Missing keys in lists.

Freelancing opportunities

- Interactive dashboards and product pages.
- Component work for existing React codebases.
- Freelance briefs for 'React developer', 'Next.js', 'dashboard', 'frontend React'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- React: Learn (official)
- Vite guide (official)
- Next.js Learn (official)

Part of **Master Freelancing: Development and IT**. Back to the chapter list on skillnavora.in · Open the full textbook. Official links in this chapter were not checked live; confirm before relying on them.